UNIX & LINUX

Why are hard links to directories not allowed in UNIX/Linux?

Asked 14 years, 2 months ago Modified 6 years, 5 months ago Viewed 82k times

161

I read in text books that Unix/Linux doesn't allow hard links to directories but does allow soft links. Is it because, when we have cycles and if we create hard links, and after some time we delete the original file, it will point to some garbage value?

If cycles were the sole reason behind not allowing hard links, then why are soft links to directories allowed?

filesystems directory symlink hard-link

Share Improve this question

Follow

edited Feb 6, 2016 at 6:12

G-Man Says 'Reinstate Monica'

24.1k 29 76 132

asked Oct 11, 2011 at 4:21

user3539

4,488 9 36 45

4 Where should `..` point to? Especially after removing the hard link to this directory, in the directory pointed to by `..`? It needs to point somewhere. – Thorbjørn Ravn Andersen Jun 24, 2013 at 22:50

3 `..` doesn't need to physically exist on any drive. It's the operating system's job to keep track of the current working directory, anyway, so it should be relatively simple to also keep a list of inodes associated with each process' cwd and refer to that when it sees a use of `..`. Of course, that would mean symlinks would need to be created with that in mind, but you already have to be careful not to break symlinks, and I don't think that additional rule would render them useless. – Parthian Shot Feb 4, 2015 at 17:28

I like this explanation. Concise and easy to read and/or skim. – Trevor Boyd Smith Dec 2, 2016 at 14:11

7 Answers

Sorted by: Highest score (default)

161

This is just a bad idea, as there is no way to tell the difference between a hard link and an original name.

Allowing hard links to directories would break the directed acyclic graph structure of the filesystem, possibly creating directory loops and dangling directory subtrees, which would make `fsck` and any other file tree walkers error prone.

First, to understand this, let's talk about inodes. The data in the filesystem is held in blocks on the disk, and those blocks are collected together by an inode. You can think of the inode as

THE file. Inodes lack filenames, though. That's where links come in.

A link is just a pointer to an inode. A directory is an inode that holds links. Each filename in a directory is just a link to an inode. Opening a file in Unix also creates a link, but it's a different type of link (it's not a named link).

A hard link is just an extra directory entry pointing to that inode. When you `ls -l`, the number after the permissions is the named link count. Most regular files will have one link. Creating a new hard link to a file will make both filenames point to the same inode. Note:

```
% ls -l test
ls: test: No such file or directory
% touch test
% ls -l test
-rw-r--r--  1 danny  staff  0 Oct 13 17:58 test
% ln test test2
% ls -l test*
-rw-r--r--  2 danny  staff  0 Oct 13 17:58 test
-rw-r--r--  2 danny  staff  0 Oct 13 17:58 test2
% touch test3
% ls -l test*
-rw-r--r--  2 danny  staff  0 Oct 13 17:58 test
-rw-r--r--  2 danny  staff  0 Oct 13 17:58 test2
-rw-r--r--  1 danny  staff  0 Oct 13 17:59 test3
            ^
            ^ this is the link count
```

Now, you can clearly see that there is no such thing as a hard link. A hard link is the same as a regular name. In the above example, `test` or `test2`, which is the original file and which is the hard link? By the end, you can't really tell (even by timestamps) because both names point to the same contents, the same inode:

```
% ls -li test*
14445750 -rw-r--r--  2 danny  staff  0 Oct 13 17:58 test
14445750 -rw-r--r--  2 danny  staff  0 Oct 13 17:58 test2
14445892 -rw-r--r--  1 danny  staff  0 Oct 13 17:59 test3
```

The `-i` flag to `ls` shows you inode numbers in the beginning of the line. Note how `test` and `test2` have the same inode number, but `test3` has a different one.

Now, if you were allowed to do this for directories, two different directories in different points in the filesystem could point to the same thing. In fact, a subdir could point back to its grandparent, creating a loop.

Why is this loop a concern? Because when you are traversing, there is no way to detect you are looping (without keeping track of inode numbers as you traverse). Imagine you are writing the `du` command, which needs to recurse through subdirs to find out about disk usage. How would `du` know when it hit a loop? It is error prone and a lot of bookkeeping that `du` would have to do, just to pull off this simple task.

Symlinks are a whole different beast, in that they are a special type of "file" that many file filesystem APIs tend to automatically follow. Note, a symlink can point to a nonexistent

destination, because they point by name, and not directly to an inode. That concept doesn't make sense with hard links, because the mere existence of a "hard link" means the file exists.

So why can `du` deal with symlinks easily and not hard links? We were able to see above that hard links are indistinguishable from normal directory entries. Symlinks, however, are special, detectable, and skippable! `du` notices that the symlink is a symlink, and skips it completely!

```
% ls -l
total 4
drwxr-xr-x  3 danny  staff  102 Oct 13 18:14 test1/
lrwxr-xr-x  1 danny  staff    5 Oct 13 18:13 test2@ -> test1
% du -ah
242M    ./test1/bigfile
242M    ./test1
4.0K    ./test2
242M    .
```

Share Improve this answer Follow

edited Feb 6, 2016 at 6:22

G-Man Says 'Reinstate Monica'
24.1k 29 76 132

answered Oct 11, 2011 at 8:28

NotAnyMore
1,968 1 14 6

12 `Allowing hard links to directories would break the directed acyclic graph structure of the filesystem` . Can you please explain more about the problem with cycles using hard links? Why is it ok with symlinks – user3539 Oct 12, 2011 at 1:13

40 They seem to have allowed it on Macs by adding cycle detection to the link() system call, and refusing to allow you to create a directory hard link if it would create a cycle. Seems to be a reasonable solution. – psusi Oct 14, 2011 at 20:08

11 @psusi mkdir -p a/b; nocheckln c a; mv c a/b; -- the nocheckln there is a theoretical ln that doesnt check for directory args, and just passes to link, and because no cycle is made, we are all good in creating 'c'. then we move 'c' into 'a/b', and a cycle is created from a/b/c -> a/ -- checking in link() is not good enough – NotAnyMore Oct 15, 2011 at 2:05

5 Cycles are *very* bad. Windows has this problem with "junctions" which are hard link directories. If you accidentally apply permissions to your whole profile, it uncovers a series of junctions that create an infinite cycle. Recursing through the directories recurses until path length limitations stop it. – doug65536 Jun 15, 2013 at 19:46

6 @WhiteWinterWolf, according to this link, they specifically added support for it for time machine, but only root is allowed to do it: superuser.com/questions/360926/... – psusi Jan 23, 2016 at 16:18

19

With the exception of mount points, each directory has one and only parent: . . .

One way to do `pwd` is to check the device:inode for '.' and '..'. If they are the same, you have reached the root of the file system. Otherwise, find the name of the current directory in the parent, push that on a stack, and start comparing '../.' with '../..', then '../../.' with '../../..', etc. Once you've hit the root, start popping and printing the names from the stack. This algorithm relies on the fact that each directory has one and only one parent.

If hard links to directories were allowed, which one of the multiple parents should `..` point to? That is one compelling reason why hardlinks to directories are not allowed.

Symlinks to directories don't cause that problem. If a program wants to, it could do an `lstat()` on each part of the pathname and detect when a symlink is encountered. The `pwd` algorithm will return the true absolute pathname for a target directory. The fact that there is a piece of text somewhere (the symlink) that points to the target directory is pretty much irrelevant. The existence of such a symlink does not create a loop in the graph.

Share Improve this answer Follow

answered Mar 23, 2012 at 23:46

Joe Inwap
522 4 3

4 Not so sure about this. If we think of `..` as being a sort of virtual hardlink to the parent, there is no technical reason that the target of the link can only have one other link to it. `pwd` would just have to use a different algorithm to resolve the path. – Benubird Mar 5, 2014 at 9:44

`..` only needs to refer to the inode of the parent. if the parent has hard links, those are just paths (names in other directories) that refer to the same inode. if `..` was a path to the parent, then it would be like a symlink. – Skaperen Aug 30, 2021 at 19:20

1 `..` links aren't needed at all and some filesystems don't create them. You can always figure out the parent from the path (prefix the cwd for relative paths). – Jim Balter Dec 26, 2021 at 13:53

@JimBalter No you can't, because you don't always know the path. `scandirat()` isn't given a full path, only a handle to the current directory and a name. This means the full path of current directory can completely change without code even knowing. This is how the shell works and explains why you can `cd` into a directory and then rename it without the shell getting into trouble. – Philip Couling Jun 7, 2023 at 12:30

^ This person is confused. Modern shells always maintain the current working directory ... that's how they can print it in the prompt. That the cwd might differ from what the .. chain yields because of renames is not relevant. – Jim Balter Jun 8, 2023 at 16:14

9

I like to add few more points about this question. Hard links for directories are allowed in linux, but in a restricted way.

One way we can test this is when we list the content of a directory we find two special directories "." and "..". As we know "." points to the same directory and ".." points to the parent directory.

So lets create a directory tree where "a" is the parent directory which has directory "b" as its child.

```
a
`-- b
```

Note down the inode of directory "a". And when we do a `ls -la` from directory "a" we can see that "." directory also points to the same inode.

```
797358 drwxr-xr-x 3 mkannan mkannan 4096 Sep 17 19:13 a
```

And here we can find that the directory "a" has three hard links. This is because the inode 797358 has three hardlinks in the name of "." inside "a" directory and name as ".." inside directory "b" and one with name "a" itslef.

```
$ ls -ali a/
797358 drwxr-xr-x 3 mkannan mkannan 4096 Sep 17 19:13 .

$ ls -ali a/b/
797358 drwxr-xr-x 3 mkannan mkannan 4096 Sep 17 19:13 ..
```

So here we can understand that hardlinks are there for directories only to connect with their parent and child directories. And so a directory without a child will only have 2 hardlink, and so directory "b" will have only two hardlink.

One reason why hard linking of directories freely were prevented would be to avoid infinite reference loops which will confuse programs which traverse filesystem.

As filesystem is organised as tree and as tree cannot have cyclic reference this should have been avoided.

Share Improve this answer Follow

answered Sep 17, 2014 at 14:04

Kannan Mohan
3,349 2 21 16

2 Good example. It cleared my doubt. So these cases are handled in a special way to avoid infinite loops. right? – G Gill Oct 7, 2014 at 1:42

1 As we have a limited way of allowing hard links for directories i.e ".." and "." we will not reach a infinite loop and so we would not require any special ways to avoid those as they will not happen :)
– Kannan Mohan Oct 8, 2014 at 3:20

@GGill There is is in fact special handling--programs that descend directory trees check the entries of a directory for the names "." and ".." don't process them, as that would result in an infinite loop.
– Jim Balter Aug 22, 2023 at 3:12

8

None of the following are the real reason for disallowing hard links to directories; each problem is fairly easy to solve:

- cycles in the tree structure cause difficult traversal
- multiple parents, so which is the "real" one ?
- filesystem garbage collection

The **real reason** (as hinted by @Thorbjørn Ravn Andersen) comes when you **delete** a directory which has multiple parents, from the directory pointed to by `..`:

What should `..` now point to ?

If the directory is deleted from its parent but its link count is still greater than `0` then there must be something, somewhere still pointing to it. You can't leave `..` pointing to nothing;

lots of programs rely on `..`, so the system would have to traverse the *entire file system* until it finds the first thing that points to the deleted directory, just to update `..`. Either that, or the file system would have to maintain a list of all directories pointing to a hard linked directory.

Either way, this would be a **performance overhead** and an **extra complication** for the file system meta data and/or code, so the designers decided not to allow it.

Share Improve this answer Follow edited Feb 13, 2016 at 14:07 answered Nov 21, 2014 at 12:54

Lqueryvg
2,009 1 18 8

3 That's easy to solve as well: keep a list of parents of a child directory, which you update when you add or remove a link to the child. When you delete the canonical parent (the target of the child's `..`), update `..` to point to one of the other parents in the list. – jathd Jan 23, 2015 at 19:29

3 I agree. Not rocket science to solve. But nonetheless a performance overhead, and it would take up a little bit extra space in the file system meta data and add complication. And so the designers went for the simple, fast approach - don't allow links to hard directories. – Lqueryvg Jan 24, 2015 at 12:54

1 Sym links to dirs "violate settled semantics and behaviours", yet they are still allowed. Some commands therefore need options to control whether sym links are followed (e.g. -L in find and cp). When a program follows '..' there is further confusion, hence the difference in output from pwd and /bin/pwd after traversing a sym link. There are no "Unix answers"; just design decisions. This one revolves around what becomes of ".." as I stated in my answer. Unfortunately, '..' isn't even mentioned in the answer that everyone else is so sheepishly voting for. – Lqueryvg May 11, 2016 at 21:18

BTW, I'm not saying I'm in favour of hard links to dirs. Not at all. I don't want my day job to be harder than it is already. – Lqueryvg May 11, 2016 at 21:19

2 It's not what POSIX says, but IMO '..' should have never been a filesystem concept, rather resolved syntactically on the paths, so that `a/..` would always mean `.`. This is how URLs work, btw. It's the browser that's resolving '..' before it even hits the server. And it works great. – ybungalobill Dec 26, 2017 at 4:31

6

Hardlink creation on directories would be unrevertable. Suppose we have :

```
/dir1
├──this.txt
├──directory
│   └──subfiles
└──etc
```

I hardlink it to `/dir2`.

So `/dir2` now also contains all these files and directories

What if I change my mind? I can't just `rmdir /dir2` (because it is non empty)

And if I recursively deletes in `/dir2` ... it will be deleted from `/dir1` too!

IMHO it's a largely sufficient reason to avoid this!

Edit :

Comments suggest removing the directory by doing `rm` on it. But `rm` on a non-empty directory fails, and this behaviour must remain, whether the directory is hardlinked or not. So you can't just `rm` it to unlink. It would require a new argument to `rm`, just to say "if the directory inode has a reference count > 1, then only unlink the directory".

Which, in turns, break another principle of least surprise : it means that removal of a directory hardlink I just created is not the same as removal of a normal file hardlink...

I will rephrase my sentence : Without further development, hardlink creation would be unrevertable (as no current command could handle the removal without being incoherent with current behaviour)

If we allow more development to handle the case, the number of pitfalls, and the **risk of data loss** if you're not enough aware of how the system works, such a development implies, is IMHO a sufficient reason to restrict hardlinking on directories.

Share Improve this answer Follow edited Jul 12, 2019 at 7:14

answered Jul 29, 2014 at 10:05

Pierre-Olivier Vares
591 4 7

1 That should not be a problem. With your case, when we create hardlink to dir2 we have to make hardlink to all the contents in dir1 and so if we rename or delete dir2 only an extra link to the inode gets deleted. And that should not affect dir1 and its content as there is atleast one link (dir1) to the inode. – Kannan Mohan Sep 17, 2014 at 13:20

5 Your argument is incorrect. You would just unlink it, not do rm -rf. And if the link count reaches 0, then the system would know it can delete all the contents too. – LtWorf Jun 12, 2017 at 12:55

1 That's more or less all `rm` does underneath anyway (unlink). See: unix.stackexchange.com/questions/151951/... This really isn't an issue, any more than it is with hardlinked files. Unlinking just removes the named reference and decrements the link count. The fact that `rmdir` won't delete non-empty directories is irrelevant - it wouldn't do that for `dir1` either. Hardlinks aren't copies of data, they are the same actual file, hence actually "deleting" the dir2 file would erase the directory listing for dir1. You would always need to unlink. – BryKKan Jul 11, 2019 at 0:49

1 You can't just unlink it like a normal file, because `rm` on a directory *don't* unlink it if it's non empty. See Edit. – Pierre-Olivier Vares Jul 12, 2019 at 7:16

This is just wrong. If hard links to directories were allowed, then of course the `rmdir` system call would not remove the inode if the link count indicated that there were other links, just as with the `unlink` system call. "without being incoherent with current behaviour" -- current behavior is that you can't hardlink to directories. Changing that of course implies that `rmdir` changes accordingly. "IMHO a sufficient reason to restrict hardlinking on directories" --- I have opinions too, but that's not what the question asks for. – Jim Balter Dec 26, 2021 at 14:02

1

This is a good explanation. Regarding "Which one of the multiple parents should .. point to?" one solution would be for a process to maintain its full wd path, either as inodes or as a string. inodes would be more robust since names can be changed. At least in the olden days, there was an in-core inode for every open file that was incremented whenever a file was opened, decremented when closed. When it reached zero it and the storage it pointed to

would be freed up. When the file was no longer open by anybody, it (The in-core copy) would be abandoned. This would maintain the path as valid if some other process moved a directory to another directory while the subdirectory was in the path of another process. Similar to how you can delete an open file but it is simply removed from the directory, but still open for any processes who have it open.

Hard-linking directories used to be freely allowed in Bell Labs UNIX, at least V6 and V7, Don't know about Berkeley or later. No flag required. Could you make loops? Yes, don't do that. It is very clear what you are doing if you make a loop. Nether should you practice knot tying around your neck while you are waiting for your turn to skydive out of a plane if you have the other end conveniently hung from a hook on the bulk-head.

What *I* hoped to do with it today was to hard-link lhome to home so that I could have /home/administ available whether or not /home was covered up with an automout over home, that automount having a symlink named administ to /lhome/administ. This enables me to have an administrative account that works regardless of the state of my primary home file system. This *IS* an experiment for linux, but I think learned at one time for the UCB based SunOS that automounts are done at the ascii string level. It is hard to see how they could be done otherwise as a layer on top of any arbitrary FS.

I read elsewhere that . and .. are not files any more in the directory either. I am sure that there are good reasons for all of this, and that much of what we enjoy (Such as being able to mount NTFS) is possible because of such things, but some of the elegance of UNIX was in the implementation. It is the benefits such as generality and malleability that this elegance provided that has enabled it to be so robust and to endure for four decades. As we loose the elegant implementations it will eventually become like Windows (I hope I am wrong!). Someone would then create a new OS which is based on elegant principles. Something to think about. Perhaps I am wrong, I am not (obviously) familiar with the current implementation. It *is* amazing though how applicable 30 year old understanding is to Linux... most of the time!

Share Improve this answer Follow

answered Jan 25, 2014 at 14:56

user57607

11 1

I think, though I may be wrong, that `.` and `..` are not hardlinks in file-system, for modern file-systems. However the file-system driver fakes them. It is these file-system that stop ups hard linking directories. For old file-systems it was possible (but dangerous). To do what you are trying, look at `mount --bind`, see also `mount --make…` and maybe containers. – ctrl-alt-delor Feb 23, 2016 at 23:03

"Hard-linking directories used to be freely allowed in Bell Labs UNIX, at least V6 and V7" -- only for superusers, who could also `unlink` directories. This was extremely dangerous and was fatal when UNIX started supporting foreign filesystems. I'm pretty sure both of these were disallowed in PWB. "some of the elegance of UNIX was in the implementation" -- much of it was necessitated by the tiny amount of memory on PDP-11s. Branches in shell scripts were done via a seek--"elegant" but bog slow. – Jim Balter Dec 26, 2021 at 14:15

0

From what I gather, the main reason is that it's useful to be able to change directory names without messing up running programs that use their working directory to reference other files. Suppose you were using Wine to run `~/.newwineprefix/drive_c/Program Files/Firefox/Firefox.exe`, and you wanted to move the entire prefix to `~/.wine` instead. If for some strange reason Firefox was accessing `drive_c/windows` by referring to `../../windows`, renaming `~/.newwineprefix` breaks implementations of `..` that keep track of the parent directory as a text string instead of an inode.

Storing the inode of a single parent directory must be simpler than trying to keep track of every path as both a text string and a series of inodes.

Another reason is that misbehaving applications might be able to create loops. Behaving applications should be able to check if the inode of the directory that's being moved is the same as the inode of any of nested directories it's being moved into, just as you can't move a directory into itself, but this might not be enforced at the filesystem level.

Yet another reason might be that if you could hardlink directories, you would want to prevent hardlinking a directory you couldn't modify. `find` has security considerations because it's used to clear files created by other users from temporary directories, which can cause problems if a user switches a real directory for a symlink while `find` is invoking another command. Being able to hardlink important directories would force an administrator to add extra tests to `find` to avoid affecting them. (Ok, you already can't do this for files, so this reason is invalid.)

Yet another reason is that storing the parent directory's inode may provide extra redundancy in case of file-system corruption or damage. If you wanted `..` to list all parent directories that hardlink to this one, so a different, arbitrary parent could be easily found if the current one is delinked, not only are you violating the idea that hard links are equal, you have to change how the file system stores and uses inodes. Having programs treat paths as a series (unique to each hardlink) of directory inodes would avoid this, but you wouldn't get the redundancy in case of file-system damage.

Share Improve this answer Follow edited Aug 14, 2018 at 22:00

answered Aug 14, 2018 at 21:17

Misaki
31 3

See similar questions with these tags.